find that they clutter the method signatures, and it's generally obvious whether a simple type is going to bind to the query string or a route value.

TIP ASP.NET Core binds to route parameters and query string values based on convention, but the only way to bind to a header value is with the `[FromHeader]` attribute.

You may be wondering what would happen if you try to bind a type to an incompatible value. What if you try to bind an `int` to the string value `"two"`, for example? In that case ASP.NET Core throws a `BadHttpRequestException` and returns a `400 Bad Request` response.

NOTE When the minimal API infrastructure fails to bind a handler parameter due to an incompatible format, it throws a `BadHttpRequestException` and returns a `400 Bad Request` response.

I've mentioned several times in this section that you can bind route values, query string values, and headers to simple types, but what *is* a simple type? A *simple type* is defined as any type that contains either of the following `TryParse` methods, where `T` is the implementing type:

```
public static bool TryParse(string value, out T result);
public static bool TryParse(
    string value, IFormatProvider provider, out T result);
```

Types such as `int` and `bool` contain one (or both) these methods. But it's also worth noting that you can create your own types that implement one of these methods, and they'll be treated as simple types, capable of binding from route values, query string values, and headers.